

Introduction to Neural Networks & Forward Propagation

Objective of Lecture

- Understand the basic concept of Neural Networks
- Formulate NN mathematically (linear transformation + activation)
- Derive equations for forward propagation
- Grasp basics of backpropagation

Motivation (Big Picture)

Why Neural Networks?

- Inspired by the brain's neurons
- Handle non-linear, high-dimensional data
- Approximate complex functions better than traditional ML

Motivation (Context Awareness)

- **Problem:** Bag-of-Words ignores word order → “*Dog bites man*” vs. “*Man bites dog*”
- **Solution:** Neural networks preserve order (RNNs, Transformers)
- **Problem:** Words have multiple meanings (*Bank* → money vs. river)
- **Solution:** Contextual embeddings (BERT, GPT)

Introduction

- Neural networks are machine learning models inspired by the structure and functioning of the human brain, capable of learning complex patterns from data.
- Why Neural Networks?

can approximate non-linear, complex functions that traditional algorithms cannot easily handle.

- Mathematical Definition of a Neural Network

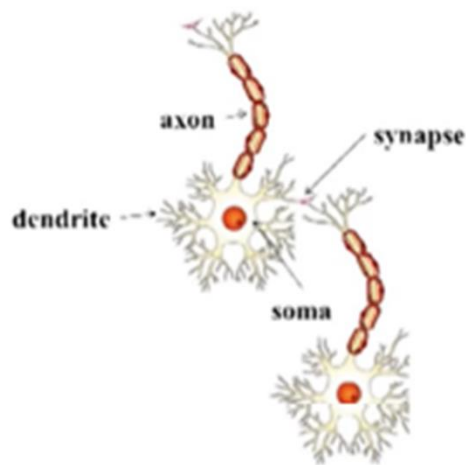
A neural network is essentially a function approximation model:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

that maps an input vector $x \in \mathbb{R}^n$ to an output vector $y \in \mathbb{R}^m$ through a composition of linear transformations and non-linear activation functions.

Biological Analogy

- Neuron: dendrites (inputs), synapse (weights), axon (output)
- Artificial neuron = simplified model



Biological Neurons*

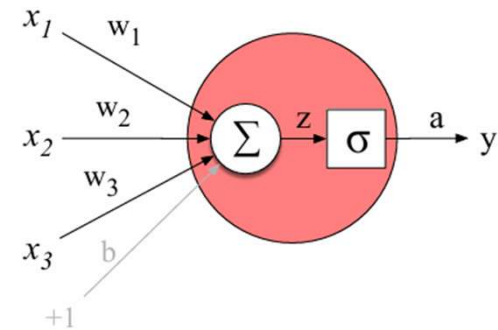
- **dendrite:** receives signals from other neurons
- **synapse:** point of connection to other neurons
- **soma:** processes the information
- **axon:** transmits the output of this neuron

Basic Elements of a Neural Network

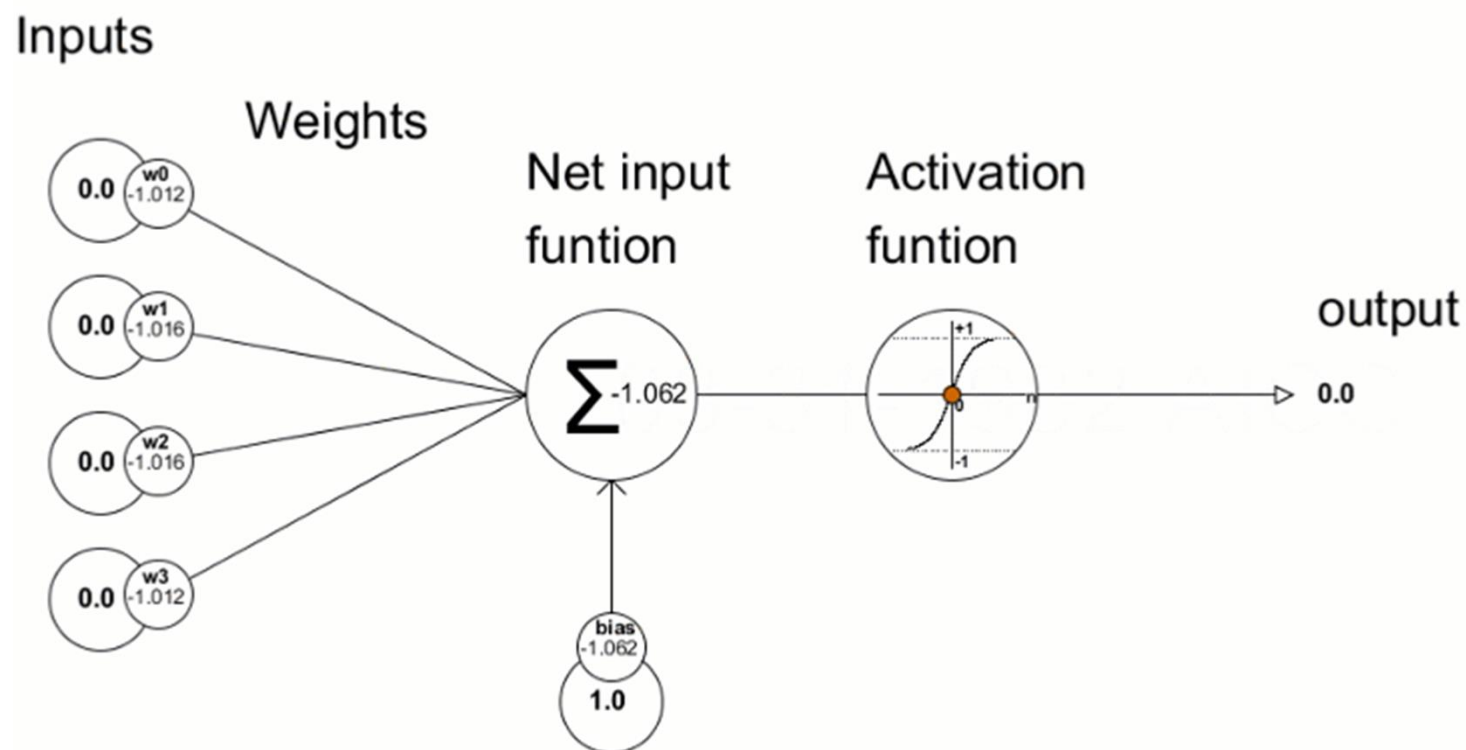
- Input layer (features).
- Hidden layers (transformations).
- Output layer (predictions).
- **Activation Functions:** Ex: sigmoid, tanh, ReLU.
- **Learning Parameters:** weights and biases.

Single Computational Unit

- Inputs ($x_1, x_2, \dots, x_n$).
- Weights ($w_1, w_2, \dots, w_n$).
- Summation function ($\sum w_i x_i$).
- Activation function (σ).
- Output (y).



Single Computational Unit: Forward Neural Network



Forward Propagation Algorithm

Input: feature vector x

Output: predicted value $\hat{y}$

1. Start with the input x (this is the first layer's activation).
2. For each layer in the network:
 - Multiply the inputs by the weights and add a bias.
 - Apply an activation function to introduce non-linearity.
 - Pass the result as input to the next layer.
3. Continue until the final (output) layer is reached.
4. The output of the last layer is the network's prediction $\hat{y}$.

Computation: Forward Propagation (Algorithm)

Given: Input vector x , weight matrices $W^{(l)}$, biases $b^{(l)}$

1. Initialize input: $a^{(0)} = x$
2. For each layer $l = 1, 2, \dots, L$:
 - $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$
 - $a^{(l)} = f(z^{(l)})$
3. Output layer $\rightarrow$ prediction

Activation Functions

Purpose: Introduce **non-linearity**. Without them, the network behaves like a linear model.

- **Common functions:**
 - **Step Function** – basic perceptron (rarely used now).
 - **Sigmoid** – squashes output between 0 and 1; suffers from vanishing gradients.
 - **Tanh** – output between -1 and 1; still gradient issues.
 - **ReLU (Rectified Linear Unit)** – most widely used; efficient and reduces vanishing gradient.
 - **Leaky ReLU / ELU** – improved versions of ReLU.
 - **Softmax** – used for classification to produce probability distribution.

Single Computational Unit

- The building block of a neural network is a single computational unit. A unit takes a set of real valued numbers as input, performs some computation on them, and produces an output.

$$z = b + \sum_i w_i x_i \quad (7.1)$$

z in terms of a weight vector $\mathbf{w}$, a scalar bias b , and an input vector $\mathbf{x}$, and we'll replace the sum with the convenient dot product:

$$z = \mathbf{w} \cdot \mathbf{x} + b \quad (7.2)$$

- Finally, instead of using z , a linear function of x , as the output, neural units apply a non-linear function f to z . We will refer to the output of this function as the activation value for the unit, a .

$$y = a = f(z)$$

popular non-linear functions $f()$ (the sigmoid, the tanh, and the rectified linear unit or ReLU)

Sigmoid Function

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (7.3)$$

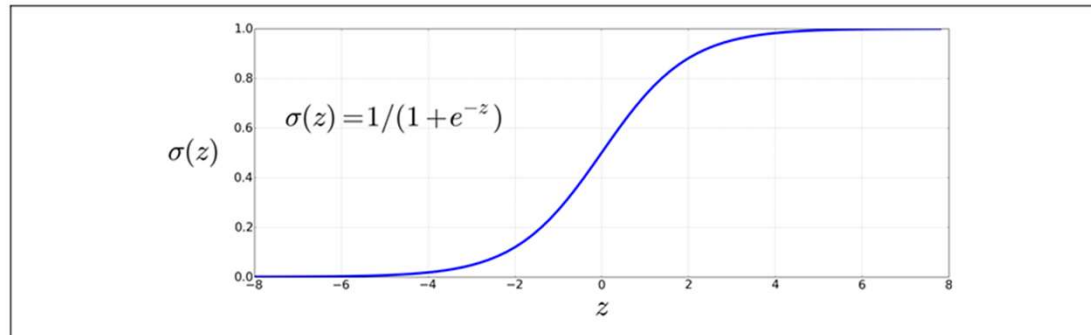


Figure 7.1 The sigmoid function takes a real value and maps it to the range $(0, 1)$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \quad (7.4)$$

Example for Sigmoid function

Let's walk through an example just to get an intuition. Let's suppose we have a unit with the following weight vector and bias:

$$\mathbf{w} = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What would this unit do with the following input vector:

$$\mathbf{x} = [0.5, 0.6, 0.1]$$

The resulting output y would be:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-(.5 * .2 + .6 * .3 + .1 * .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

tanh vs ReLU

◆ tanh Function

- Sigmoid is not commonly used in practice.
- **tanh** is almost always better.
- Output range: **-1 to +1**

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

◆ ReLU Function

- **Simplest & most commonly used** activation function.
- ReLU = Rectified Linear Unit
- Formula:

$$y = \text{ReLU}(z) = \max(z, 0)$$

- Passes positive values, outputs 0 otherwise.

Example – Forward Neural Network

- Input: $x = [1, 2]$
- Hidden Layer: 2 neurons, ReLU activation
 - $h_1 : w = [0.2, 0.4], b = 0.1$
 - $h_2 : w = [0.5, 0.6], b = 0.2$
- Output Layer: 1 neuron, Sigmoid activation
 - $w = [0.3, 0.8], b = 0.5$

Hidden Layer Computation

Pre-activation:

$$z_1^{(1)} = 0.2(1) + 0.4(2) + 0.1 = 1.1$$

$$z_2^{(1)} = 0.5(1) + 0.6(2) + 0.2 = 1.9$$

Sigmoid Activation:

$$a_1^{(1)} = \sigma(1.1) = \frac{1}{1 + e^{-1.1}} \approx 0.75$$

$$a_2^{(1)} = \sigma(1.9) = \frac{1}{1 + e^{-1.9}} \approx 0.87$$

Output Layer & Final Prediction

Pre-activation:

$$z^{(2)} = 0.3(0.75) + 0.8(0.87) + 0.5 \approx 1.41$$

Sigmoid Activation:

$$\hat{y} = \sigma(1.41) = \frac{1}{1 + e^{-1.41}} \approx 0.80$$

✅ Final Output = 0.80 (80% probability)

Thank You